

Skip List – Insert Element

–Time Complexity Analysis

Program:

```
void insertElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;

    // Array to keep track of the path we took (nodes needing pointer
    updates)
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Start from the highest current level and move down
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    // Move to level 0 and look at the next node
    current = current->forward[0];

    // If the key doesn't already exist, insert it
    if (current == NULL || current->key != key) {
        int rlevel = randomLevel();

        // If the random level is greater than the list's current max
        level,
        // initialize the update array for the new levels to point to
        the header.
        if (rlevel > lst->level) {
            for (int i = lst->level + 1; i <= rlevel; i++) {
                update[i] = lst->header;
            }
            lst->level = rlevel;
        }
    }
}
```

```

    // Create the new node and rewire pointers
    struct Node *n = createNode(key, rlevel);
    for (int i = 0; i <= rlevel; i++) {
        n->forward[i] = update[i]->forward[i];
        update[i]->forward[i] = n;
    }
    cout << "Successfully inserted key " << key << endl;
} else {
    cout << "Key " << key << " already exists in the list." << endl;
}
}
int main() {
    // Seed random number generator for random levels
    srand((unsigned)time(0));
    struct SkipList *list = createSkipList();
    int choice, key;

    while (true) {

        cout << "\n==== Skip List Menu =====" << endl;
        cout << "1. Insert" << endl;
        cout << "Enter your choice: ";

        if (!(cin >> choice)) {
            cout << "Invalid input. Exiting." << endl;
            break;
        }

        switch (choice) {

        case 1:
            cout << "Enter key to insert: ";
            cin >> key;
            insertElement(list, key);
            break;
        }
    }
    return 0;
}

```

Time Complexity Analysis

1. Best Case Time Complexity: $O(1)$

The best – case scenario occurs when the element to be inserted is smaller than the very first element in the list (right after the header), or when the list is completely empty.

- *The traversal loop begins at the highest level but immediately hits NULL or a key larger than the target at every single level.*
- *The search halts immediately without horizontally traversing any nodes.*
- *Time Complexity: $O(1)$.*

True Absolute Best Case Scenario

For this to happen in a Skip List insertion, two specific conditions must be met simultaneously:

1. ***The insertion point is at the very front:*** *The list is either completely empty, or the key you are inserting is smaller than the very first element in the list.*
2. ***The coin flip fails immediately:*** *The `randomLevel()` function generates an `rlevel` of 0 on its very first try.*

1. Initialization

```
struct Node *current = lst->header;  
struct Node *update[MAX_LEVEL + 1];  
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

Behavior: This section always executes normally. The **update** array is allocated on the stack and zeroed out. Because **MAX_LEVEL** is a small constant, this takes a tiny, constant amount of time (**$O(1)$**).

2. The Search Phase (Ducking the Horizontal Traversal)

```
for (int i = lst->level; i >= 0; i--) {  
    while (current->forward[i] != NULL && current->forward[i]->key < key) {  
        current = current->forward[i];  
    }  
    update[i] = current;  
}
```

Recalling the **for loop** behavior:

$\text{for (int } i = \text{lst} \rightarrow \text{level}; i \geq 0; i--)$: This initiates a top – down search.

Instead of searching one by one from the bottom, it starts at the highest current level of the list (**$\text{lst} \rightarrow \text{level}$**) and drops down step – by – step to level 0 (**$i--$**).

This is what gives a skip list its speed, allowing it to use the upper levels as "express lanes" to skip over large chunks of data.

1st → Level 3

2nd → Level 2

3rd → Level 1

4th → Level 0

Goes top – down vertically.

while (current-> forward[i] != NULL && current-> forward[i]-> key < key):

At each level, the program looks ahead to the next node. It checks two critical things before moving:

1. *!= NULL: Does a next node actually exist, or have we reached the end of the list?*

2. *→ key < key: Is the next node's key smaller than the one we are trying to insert?*

- *current = current → forward[i]; If the next node exists and has a smaller key, the current pointer moves horizontally to the right. This while loop continues moving the pointer rightward until it hits a node that is larger, or hits the end of the list.*

- *update[i] = current; This is the most crucial part for the actual insertion. When the current pointer can no longer move forward on level i, it stops. The program then records this exact node into the update array (update[i] = current) before the for loop forces the pointer to drop down to the next level.*

Hence, while (current-> forward[i] != NULL && current-> forward[i]-> key < key): moves horizontally.

Now coming back to the Time Complexity:

Behavior: The **outer for** loop must still run to drop down from the **top level to level 0**. However, the **inner while loop** is completely ducked.

- Because the new key belongs at the very front of the list, the condition **$current \rightarrow forward[i] \neq NULL \ \&\& \ current \rightarrow forward[i] \rightarrow key < key$** evaluates to **False** on the very first check at every single level.
- The pointer **current** never moves horizontally. Instead, the **update** array at every level **i** is simply populated with the address of the **header** node.

The Inner while Loop is 100% Ducked: This is the horizontal traversal loop : **$while (current \rightarrow forward[i] \neq NULL \ \&\& \ current \rightarrow forward[i] \rightarrow key < key)$** . Because the next node is either **NULL** (empty list) or its key is larger than our target key, this condition evaluates to **False** on the very first check at every single level. The program entirely bypasses the inside of this loop, meaning zero horizontal steps are taken.

If inserting 20 before 50: The telescope sees the node with 50. Because 50 is not less than 20, the condition fails immediately. **$current \rightarrow forward[i] \rightarrow key$** is the Next Node's Key and **Key** is our target key, eg: Next Node's Key : 50 < Key : 20 , the condition sets to false.

The Outer for Loop MUST Still Run: This is the vertical drop-down loop: **$for (int i = lst \rightarrow level; i \geq 0; i--)$** .

The program cannot duck this loop. Even though no horizontal movement is needed, the program must still drop down level by level to populate the **update** array with the header node's address. It has to tell the update array, "Hey, at level 3, the predecessor is the header. At level 2, the predecessor is the header," and so on, so that the new node can be wired correctly.

Note: *MAX_LEVEL* is constant and *lst → level* is also a constant and

lst → level $\leq$ *MAX_LEVEL* ≤ 3 , hence for loop runs from *lst → level* to 0 takes constant amount of time, *O(c)* or *O(1)*.

3. Duplicate Key Validation

```
// Move to level 0 and look at the next node
current = current->forward[0];

// If the key doesn't already exist, insert it
if (current == NULL || current->key != key) {
```

Behavior: *current* moves one step forward on level 0.

Because the list is empty (so *current* is *NULL*) or the new key is smaller than the first key (so *current → key! = key*), this if condition evaluates to *True*, and the program proceeds to insert.

4. Level Generation

```
int rlevel = randomLevel();
```

Behavior: In this absolute best – case scenario, the random coin flip inside *randomLevel()* fails on its very first try. The function immediately breaks its internal loop and returns an *rlevel* of 0. This takes instantaneous *O(1)* time.

```
int randomLevel() {
    int lvl = 0;
    // 50% probability to increase the level
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {
        lvl++;
    }
    return lvl;
}
```

rand() % 2 $\neq 0$, hence, while loop condition fails and return *lvl* = 0.
hence, *rlevel* = *randomLevel()* = 0.

5. Height Adjustment (Completely Skipped)

```
if (rlevel > lst->level) {
    for (int i = lst->level + 1; i <= rlevel; i++) {
        update[i] = lst->header;
    }
    lst->level = rlevel;
}
```

- **Behavior:** Because our lucky *rlevel* is 0, and the list's current level (*lst->level*) is at least 0, the condition (*0 > lst->level*) evaluates to False.
- This entire block of code – including the *for* loop and the reassignment of *lst->level* – is completely skipped, saving execution time.

6. Node Creation and Rewiring

```
struct Node *n = createNode(key, rlevel);
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
```

- **Behavior:** *createNode* allocates memory for the new node with an array size supporting just *level 0*.
- Because *rlevel* is 0, the rewiring *for* loop runs exactly one time (*for i = 0*).
- It takes the *update[0]* address (which we know is the *header*), points the new node's level 0 pointer to whatever the header was pointing to, and then points the header's level 0 pointer to the new node.

The loop then terminates instantly.

In the true absolute best – case scenario, the program effortlessly bypasses the horizontal while loop entirely, skips the height adjustment if block completely, and executes the final rewiring for loop exactly once. The entire insertion collapses into an ultra – efficient sequence of constant – time $O(1)$ operations with the absolute bare minimum number of CPU instructions.

2. Worst Case Time Complexity

The worst – case scenario is a structural failure of the skip list's probabilistic balancing.

Imagine a situation where the random number generator consistently flips tails(fails to promote)for every single node inserted, meaning every node in the list only exists at level 0.

- *The skip list devolves into a standard, **standard singly linked list**.*
- *If we want to insert a key at the very end of this list, the search algorithm must start at the header and visit every single node one by one at level 0.*
- **Math:** *If we insert an element that belongs at the very end of this list, the algorithm has no upper levels to skip across. It must evaluate every single node one by one. For n elements, this requires n comparisons.*
- **Result:** *The complexity degrades to $O(n)$.*

Worst Case Analysis

1. Initialization

```
struct Node *current = lst->header;  
struct Node *update[MAX_LEVEL + 1];  
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

Behavior: This setup phase behaves exactly as it normally would.

The **update** array is allocated to hold 4 elements (since **MAX_LEVEL** is 3) and is wiped clean using `memset` to ensure no wild pointers exist.

2. The Search Phase (The $O(n)$ Bottleneck)

```
for (int i = lst->level; i >= 0; i--) {  
    while (current->forward[i] != NULL && current->forward[i]->key <  
key) {  
        current = current->forward[i];  
    }  
    update[i] = current;  
}
```

- **Behavior:** This is where the disaster happens. Because every previously inserted node had a level of 0, the skip list's overall maximum height (`lst->level`) never grew. It remains at 0.
- Consequently, the `for` loop executes exactly once (`for i = 0`). There are no "express lanes" to drop down through.
- The **inner while** loop must now do all the heavy lifting. The condition `current->forward[0] != NULL` ensures the pointer moves until it hits the end of the list.

- Because the new target **key** belongs at the very end, the condition **current → forward[0] → key < key** remains **True** for every single node currently in the list.
- The command **current = current->forward[0]** executes **n** times, forcing the pointer to physically step through all **n** nodes **one by one**. This single line is what degrades the time complexity to **O(n)**.
- Finally, the **loop halts** when it hits **NULL**, and **update[0]** records the very last node in the list.

3. Validation

```
current = current->forward[0];

// If the key doesn't already exist, insert it
if (current == NULL || current->key != key) {
```

- **Behavior:** After traversing the entire list, stepping forward one final time makes **current** step off the **last node** and land exactly on **NULL**.
- The **if** condition evaluates (**current == NULL**). Since this is **True**, the program validates that the key does not already exist and proceeds to insert it.

4. Level Generation and Height Adjustment

```
int rlevel = randomLevel();

// If the random level is greater than the list's current max
level,
// initialize the update array for the new levels to point to
the header.
if (rlevel > lst->level) {
    for (int i = lst->level + 1; i <= rlevel; i++) {
        update[i] = lst->header;
    }
    lst->level = rlevel;
}
```

- **Behavior:** The program rolls the dice for the new node.
- If this new node finally gets lucky and generates an **rlevel** greater than 0 (e.g., **rlevel = 2**), the if block executes.
- The **update** array slots for **levels 1 and 2** are anchored directly to the **lst → header**, and the global list height is finally raised, effectively fixing the "**flat list**" problem for future insertions!

5. Node Creation and Rewiring

```
struct Node *n = createNode(key, rlevel);
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
```

- **Behavior:** The new node is created in memory.
 - The rewiring loop executes. For $i = 0$, it takes the pointer from `update[0]` (which we recorded earlier as the former last node of the list) and points it directly to our newly created node.
- The new node is successfully appended to the absolute tail end of the list.

For new $rlevel$ (eg: $rlevel = 2$), it goes like this : –

1. Declaring the New Height

In the first block you shared, the program realizes that the new node's random level ($rlevel = 2$) is bigger than the list's current maximum level ($lst \rightarrow level = 0$).

- It executes $lst \rightarrow level = rlevel$; which officially upgrades the entire skip list from a flat 1 – story road to a 3 – story building (Levels 0, 1, and 2).

2. Setting Up the "Starting Line"

Because Levels 1 and 2 are brand new, there are no existing nodes on these "upper floors" to connect to.

- The *if* block handles this by setting `update[1] = lst->header` and `update[2] = lst->header`.
- This tells the program: "For Levels 1 and 2, the only thing that exists to connect to so far is the starting gate (the header node)."

3. Building the Tall Node

The program calls `struct Node *n = createNode(key,rlevel);`.

- This creates a physical node in memory that is tall enough to hold three forward pointers (one for Level 0, one for Level 1, and one for Level 2).

4. Stringing the Pointers (The Magic)

The final *for* loop is where the new levels are actually wired into existence:

- At Level 0 (*i* = 0): The new node links up to the old "tail end" of the list, just like a normal linked list addition.
- At Level 1 (*i* = 1): The program looks at `update[1]`, which we explicitly set to the header. It takes the header's Level 1 pointer and strings it all the way across the list, skipping over all the short, flat nodes, and attaches it directly to our newly created tall node.

Dynamic Node Sizes: Unlike a standard linked list where every single node is the exact same size, nodes in a Skip List are dynamically sized based on their randomly assigned level (*rlevel*).

A Short Node (Level 0): If a node rolls an *rlevel* of 0, the *createNode* function allocates an array with only 1 pointer slot (*forward[0]*). Because it only has one connection pointing horizontally along the bottom floor, we can visualize this node as being "short" or "flat."

A Tall Node (Level 2 or 3): If a node gets lucky and rolls an *rlevel* of 2, the *createNode* function allocates an array with 3 pointer slots (*forward[0]*, *forward[1]*, and *forward[2]*). Because this node has to hold connections for the *ground floor* and the *upper express lanes*, it physically takes up more memory.

- **At Level 2 ($i = 2$):** It does the exact same thing. It takes the header's **Level 2 pointer** and strings a massive bridge directly to the new node.

By the time this loop finishes, Level 1 and Level 2 officially exist!

They act as giant express lanes bridging directly from the starting header to our newly inserted node at the very end of the list.

"Because every node is generated with a height of 0, the structural advantage of the skip list is completely lost. The array of forward pointers inside every node contains only a single valid link (at level 0) pointing to the immediate next node. The algorithm is forced to traverse a single, flat chain of nodes one by one."

Alternative Worst Case

The "All Heads" Scenario (The Express Lane Traffic Jam)

The exact opposite of the all tails scenario can also destroy the efficiency of a Skip List.

- *Imagine the random number generator is completely broken in the other direction, consistently flipping **heads** and promoting every single node to the absolute maximum level (e.g., **MAX_LEVEL = 3**).*
- *In this scenario, every single node in the list has a pointer at **level 3**.*

- When we search for an item, the algorithm starts at the highest level (*1st → level, which is 3*). Because every node exists on **level 3**, the algorithm cannot skip over anything. It is forced to traverse **horizontally** across every single node on the top floor.
- The **express lane** is completely clogged, turning the top level into a **standard singly linked list**. The performance degrades to **$O(n)$** , proving that some arrangements of randomly generated levels will give poor execution times.

The above alternative worst case explanation as follows:

The "Express Train" Analogy

How a Skip List SHOULD Work:

- **Level 0 is the Local Train.** It stops at every single station (every node) on the track. It is slow, but it guarantees we don't miss your stop.
- **Level 3 is the Express Train.** It only stops at major city hubs. When we ride the Express Train, we bypass (or skip) dozens of tiny local stations, covering massive distances in a single jump.

How the All Heads Scenario Ruins It: Now, imagine a scenario where the random number generator promoted every single node to Level 3.

- In our analogy, this means the city planners accidentally upgraded every tiny local station into a major city hub.
- Because every single node is now a Level 3 station, the Express Train is forced to stop at Node 1, then Node 2, then Node 3, and so on.

The Code Mechanics Behind the Traffic Jam

Let's look at what the pointers are actually doing when this happens.

When you start your search, the for loop begins at the top floor: $i = 3$.

*The algorithm looks at the **current** $\rightarrow$ **forward[3]** pointer, hoping to make a massive jump across the list.*

- ***Normally:*** The **forward[3]** pointer of the header might point to the 8th node in the list. We just successfully skipped 7 nodes in a single step!
- ***The Worst Case:*** Because every node has a Level 3 pointer, the header's **forward[3]** pointer just points to Node 1. Node 1's **forward[3]** pointer points to Node 2. Node 2's **forward[3]** points to Node 3.

*The algorithm is trapped on the top floor, but the top floor has become exactly as crowded as the bottom floor. The search is forced to evaluate every single node one by one horizontally across Level 3, resulting in a painfully slow **$O(n)$** execution time.*

*Here trapping can mean **Dead Lock** situation, but its not:*

For example: –

1. Starting at the Top Floor ($i = 3$)

- The **for** loop starts at $i = 3$. The current pointer starts at the **header**.
- The **while** loop looks ahead at **Level 3**. It sees **Node 10**. Since **$10 < 100$** , it steps forward to **Node 10**.
- It looks ahead again on **Level 3**. It sees **Node 20**. Since **$20 < 100$** , it steps forward to **Node 20**.

- It looks ahead again on **Level 3**. It sees **Node 30**. Since **30 < 100**, it steps forward to **Node 30**.
- It looks ahead one last time on **Level 3**. It sees **NULL** (the end of the list). The **while** loop safely and successfully terminates.
- **Result:** The algorithm just did **O(n)** work crossing the entire list on the top floor.

2. Dropping Down (*i* = 2, 1, 0)

- Now the **for** loop drops down to **i = 2**.
- The **current** pointer is already sitting on **Node 30** at the very end of the list.
- The **while** loop looks ahead on **Level 2**. Because **Node 30** is the last node, **forward[2]** is **NULL**. The **while** loop terminates instantly with zero steps taken.
- This repeats for **Level 1** and **Level 0**. The **while** loop safely skips execution because we are already at the end of the line.

3. Finishing the Program

- The loop structure completes successfully.
- The program proceeds to create the new node for 100 and wires it into the list.

"Because every single node in the structure has been promoted to the maximum level, the forward pointers at the highest level can no longer 'skip' over anything. Instead of jumping across large sections of data, the top – level pointer of each node simply points to its immediate neighbor, forcing a linear O(n) traversal across the top floor."

Why It Is Not a Deadlock

A deadlock means the program physically cannot move to the next line of code. As the trace above shows, the algorithm never stops running. The **while** loop successfully processes **Node 10**, then **Node 20**, then **Node 30**, and then it correctly exits the loop. It just had to perform **n** horizontal comparisons on **Level 3** to get there.

Average Time Complexity

In a skip list, the total time to insert a new element is the sum of three distinct phases:

1. *The Search Phase: Finding the correct insertion point.*
2. *The Coin Flip Phase: Generating the random level for the new node.*
3. *The Splicing Phase: Rewiring the pointers.*

Phase 1: The Search Phase

Let n be the total number of elements in the skip list.

Let p be the probability that a node at level k is promoted to level $k + 1$ (typically, 50%, so, $p = 0.5$).

Step 1A: Calculate the Expected Maximum Height(H)

To know how far up we have to climb, we first need to determine the expected maximum height of the entire skip list.

- *The probability that any single node reaches level k is p^k .*
- *Let N_k be the number of nodes at level k . The expected number of nodes at level k across the whole list is:*

$$E[N_k] = n \times p^k$$

The maximum height H of the skip list is defined as the level where we expect to find exactly 1 node. Therefore, we set $E[N_k] = 1$:

$$n \times p^H = 1$$

Solving for H :

$$p^H = \frac{1}{n}$$

Putting $\log_p$ at both the side we get:

$$\log_p(p^H) = \log_p\left(\frac{1}{n}\right)$$

$$H = \log_p(n^{-1}) \left[\log_b x^y = y \times \log_b x, \log_b b = 1 \text{ and } \frac{1}{n} = n^{-1} \right]$$

$$H = -1 \times \log_p n \left[\log_b x^y = y \times \log_b x \right]$$

$$H = -\log_p n$$

To get rid of that negative sign, we use the Change of Base Formula

$\left(\log_b(x) = \frac{\log_c(x)}{\log_c(b)} \right)$ to change the base from p to $\frac{1}{p}$:

$$H = -\log_p(n) = -\left(\frac{\log_{\frac{1}{p}}(n)}{\log_{\frac{1}{p}}(p)} \right)$$

Look at the denominator : $\log_{\frac{1}{p}}(p)$. Because p is the exact

inverse of $\frac{1}{p}$ (meaning $p = \left(\frac{1}{p}\right)^{-1}$), this denominator evaluates

to -1 .

$$H = -\log_p(n) = -\left(\frac{\log_{\frac{1}{p}}(n)}{\log_{\frac{1}{p}}\left(\frac{1}{p}\right)^{-1}} \right)$$

$$H = - \left(\frac{\log_{\frac{1}{p}}(n)}{-1 \times \log_{\frac{1}{p}}\left(\frac{1}{p}\right)} \right)$$

$$H = - \left(\frac{\log_{\frac{1}{p}}(n)}{-1} \right)$$

$$H = \log_{\frac{1}{p}}(n)$$

Alternative calculation:

$$p^H = \frac{1}{n}$$

$$\Rightarrow n = \frac{1}{p^H}$$

We can also write it as:

$$\Rightarrow n = \left(\frac{1}{p}\right)^H \quad [as, 1^H = 1]$$

*The fundamental definition of a logarithm is that if $b^y = x$, then $\log_b(x) = y$. Here our base $\left(\frac{1}{p}\right)$, our exponent is H , and our result is n . By simply rewriting it in **log** form, we instantly get:*

$$H = \log_{\frac{1}{p}}(n)$$

if $p = 0.5$, the expected maximum height is:

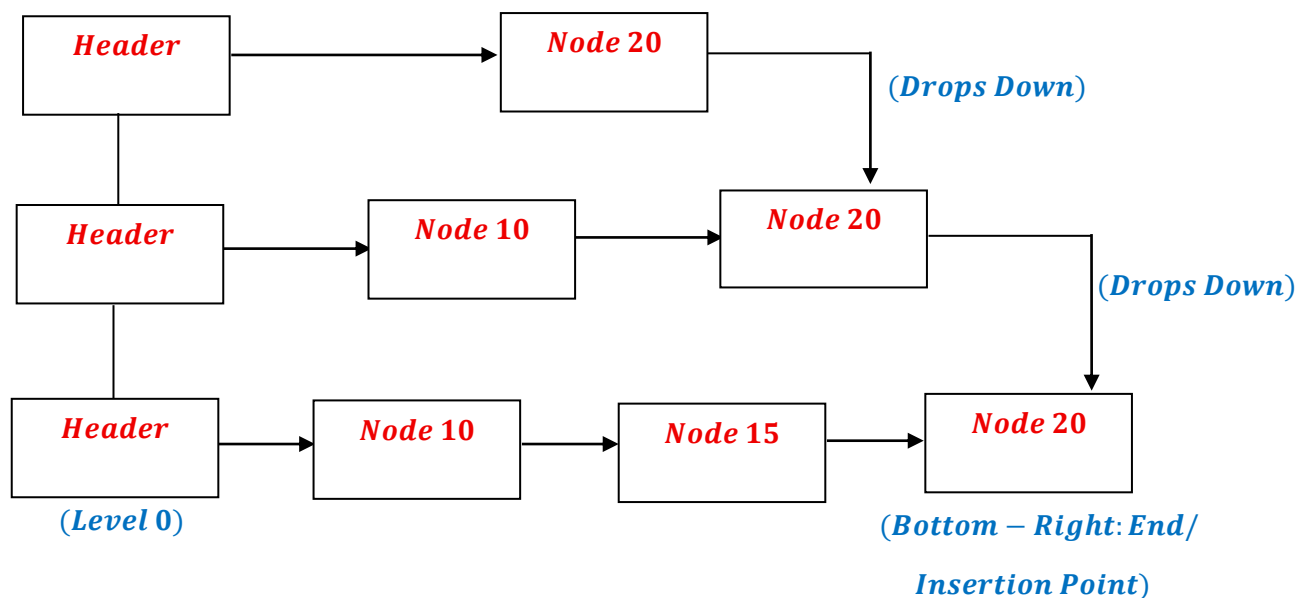
$$H = \log_{\frac{1}{0.5}}(n)$$

$$H = \log_{\frac{1}{\frac{1}{2}}}(n)$$

$$H = \log_2(n)$$

Step 1B : Calculate the expected Horizontal Steps per Level

Now, imagine walking backward from the bottom – right insertion point . At any given node on level `k`, there are only two ways the forward search could have reached it:



→ We place ourself at the final destination [Node 20] at Level 0.

- *Instead of moving Right, you move Left.*
- *Instead of dropping Down, you climb Up.*

[illegible]

- **Forward Search:** Moves RIGHT, then drops DOWN, then moves RIGHT, then drops DOWN...until it hits the bottom.
- **Reverse Analysis (Rewind):** Moves LEFT, then climbs UP, then moves LEFT, then climbs UP...until it hits the top – left start.

1. It dropped down from the above: This happens if the node exists at level $k + 1$. The probability of this p . In reverse, this means we move UP.

Scenario A: The Code Dropped Down (Reverse = Move UP)

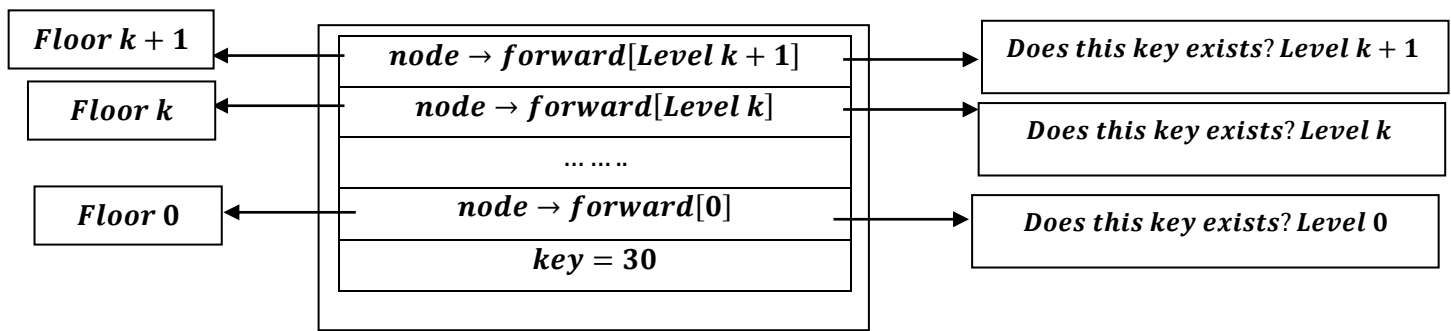
```
struct Node *current = lst->header;
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    ...
}
```

- **The Code:** Imagine the node we are standing on was lucky during `randomLevel()` and was built with a level of $k + 1$ or higher. Because this node physically exists on the floor above us, our for loop ($i--$) must have used this exact node to drop down from level $k + 1$ to level k .
- **The Reverse Math:** Since the forward code dropped DOWN at this node, our backward analysis climbs UP to level $k + 1$.
- **The Probability:** What are the odds that `randomLevel()` gave this node a height of at least $k + 1$? Exactly p (which is 50% in our code).

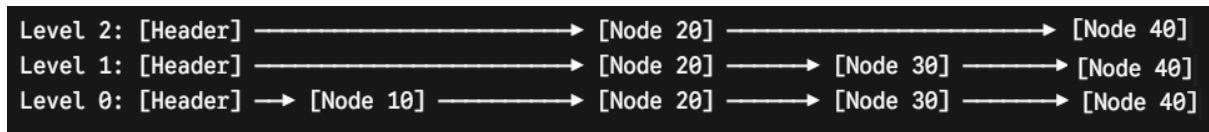
```
#define MAX_LEVEL 3
```

```
lst->header = createNode(-1, MAX_LEVEL);
```

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));
```



if



Then, **level 2** is here **level $k+1$** .

Normal Forward Search – Which is defined by Program

Step 1: Forward Code Execution

1. Iteration $i = 2$ (Level 2)

- **current starts at Header.**
- **while loop runs:**
 - **Header $\rightarrow$ forward[2] is Node 20 ($20 < 35$).current becomes Node 20 (Moved RIGHT).**
 - **Node 20 $\rightarrow$ forward[2] is Node 40 ($40 < 35$ is False).while loop stops.**
- **update[2] = Node 20.**

- The for loop executes $i--$:
 - i becomes 1.
 - Notice: **current** is still **Node 20**. The pointer did not move to another node; only the floor i changed from 2 to 1.
 - This is the vertical drop from (**Node 20, Level 2**) → (**Node 20, Level 1**).

2. Iteration $i = 1$ (Level 1)

- Entering with: **current** = **Node 20**, $i = 1$.
- while loop runs:
 - **Node 20** → **forward[1]** is **Node 30** ($30 < 35$). **current** becomes **Node 30** (**Moved RIGHT**).
 - **Node 30** → **forward[1]** is **Node 40** ($40 < 35$ is **False**). while loop stops.
- **update[1]** = **Node 30**.
- The for loop executes $i--$:
 - i becomes 0.
 - Notice: **current** is still **Node 30**.
 - This is the vertical drop from (**Node 30, Level 1**) → (**Node 30, Level 0**).

3. Iteration $i = 0$ (Level 0)

- Entering with: **current** = **Node 30**, $i = 0$.
- while loop runs:
 - **Node 30** → **forward[0]** is **Node 40** ($40 < 35$ is **False**). while loop stops immediately.
- **update[0]** = **Node 30**.

Step 2: Now Look at the Backward Path

Look at what the code recorded in **update[]**:

- **update[2]** = **Node 20**
- **update[1]** = **Node 30**
- **update[0]** = **Node 30**

*In program , it runs strictly forward search (Top – left to Bottom right),
program doesn't run from Bottom ,step – up i. e. Rewind or Backward Walk
, Backward Walk is only done for mathematical purpose.*

Let's rewind state – by – state starting from Level 0:

Rewind from (Node 30, Level 0):

- *The state immediately before (Node 30, $i = 0$) was (Node 30, $i = 1$).*
- *Rewinding: The same node at the previous level $\Rightarrow$
Step UP to (Node 30, Level 1).*

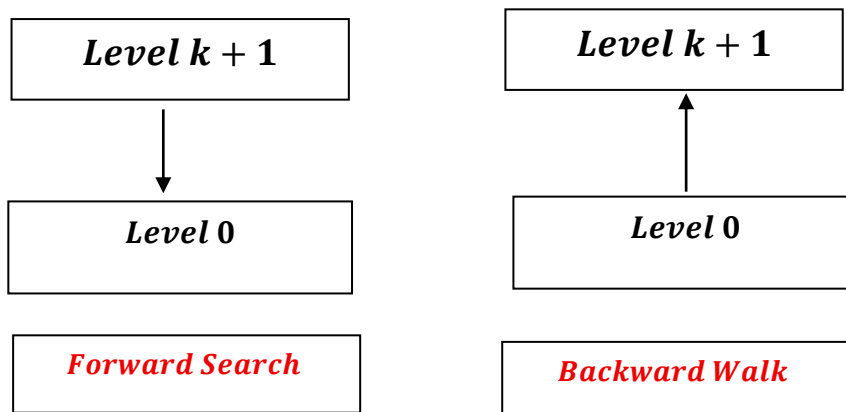
Rewind from (Node 30, Level 1):

- *The state immediately before was (Node 20, $i = 1$).*
- *Rewinding: A different node on the same level
 $\Rightarrow$ Step LEFT to (Node 20, Level 1).*

Rewind from (Node 20, Level 1):

- *Node 20 exists on Level 2 ($update[2] == Node\ 20$).*
- *Rewinding: $\Rightarrow$ Step UP to (Node 20, Level 2).*

*That is : **Backward Walk** ,also walks from Level 0 $\rightarrow$ Level $k + 1$.*



Phase 1: The Search Phase

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

Phase 2: The Coin Flip & Insertion Phase

```
if (current == NULL || current->key != key) {
    int rlevel = randomLevel(); // <--- 1. THE COIN FLIP HAPPENS
    HERE
    ...
    // 2. CREATE AND ATTACH POINTERS FOR EACH LEVEL REACHED
    struct Node *n = createNode(key, rlevel);
    for (int i = 0; i <= rlevel; i++) {
        n->forward[i] = update[i]->forward[i];
        update[i]->forward[i] = n;
    }
}
```

How randomLevel() works under the likelihood:

```
int randomLevel() {
    int lvl = 0;
    // 50% probability to increase the level
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {
        lvl++;
    }
    return lvl;
}
```

<i>Returned lvl</i>	<i>Formula</i>	<i>Reason</i>	<i>Probability</i>
<i>lvl = 0</i>	<i>$1 - p$</i>	<i>rand()%2 = 1, very first coin flipped failed.</i>	<i>50% = 0.5</i>
<i>lvl = 1</i>	<i>$p(1 - p)$</i>	<i>rand()%2 = 1, at 2nd time coin flipped failed.</i>	<i>50% × 50% = 0.5² = 0.25 = 0.25 × 100 = 25%</i>
<i>lvl = 2</i>	<i>$p^2(1 - p)$</i>	<i>rand()%2 = 1, at 3rd time coin flipped failed.</i>	<i>50% × 50% × 50% = 0.5³ = 0.125 = 0.125 × 100 = 12.5%</i>
<i>lvl = 3</i>	<i>$p^3(1 - p)$</i>	<i>rand()%2 = 1, at 4th time coin flipped failed.</i>	<i>50% × 50% × 50% × 50% = 0.5⁴ = 0.0625 = 0.0625 × 100 = 6.25%</i>

When **Coin** flipped failed , it generates **Tails**(**rand()%2 = 1**) i. e. **1 - p**

When **Coin** flipped passed , it generates **Head**(**rand()%2 = 0**)i. e. **p**

→ So, when returned level 0 , 1st flipped failed so : **1 - p**

→ When returned level 1 , 1st flipped passed generated `p` but 2nd flipped failed
generates `1 - p` ⇒ **$p(1 - p)$**

→ When returned level 2 , 2 times flipped passed generated `p` but 3rd flipped failed
generates `1 - p` ⇒ **$p^2(1 - p)$**

And so on

There fore ,we find out that:

→ *Probability of reaching Level 0: 100% [i. e. Reching Level 0 is guaranteed]*

→ *Probability of reaching Level 1: 50% (p)*

→ *Probability of reaching Level 2: 25% ($p \times p = p^2$)*

→ *Probability of reaching Level 3: 12.5% ($p \times p \times p = p^3$)*

Conditional Probability (When ALREADY standing on Level k)

During the search analysis,if we are already standing on a node at Level k .That means this node already had k successful coin flips in the past.

Now, : What is the chance this node also reaches Level $k + 1$?

Because coin flips are independent (memoryless), it only depends on one single next coin flip:

- *Next flip is Heads (50% or p): The node reaches Level $k + 1$.*

How p ? Now Mathematical terms:

$$\begin{aligned} \text{Pr(Reaches Level } k + 1 \mid \text{Already on Level } k) &= \frac{\text{Pr(Reaches Level } k + 1)}{\text{Pr(Reaches Level } k)} = \frac{p^{k+1}}{p^k} \\ &= p = 0.5 \end{aligned}$$

Where , p represents probability $\left(\text{i. e. } \frac{\text{Probability when reaches level } k + 1}{\text{Probability when we are Already standing at level } k} \right)$

$\therefore$ Reaching Level ' $k + 1$ ' ,while moving through horizontal step is: ' p '

2. It moved horizontally: This happens if the node doesnot exist at level ` $k + 1$ '. The probability of this ` $1 - p$ '. In reverse . this means we move LEFT .

This happens because , say,

→ we have **Level ` $k + 1$ `** , say **3** .

→ But **rand() %2** fails and gives us **Level ` $k$ `** , i. e. returns 2 , i. e. it will go at most **level 2** to insert our node.

Suppose **Node 30** was created , **randomLevel()** failed to promote it in **Level $k + 1$** (eg: **Level 3**).

As,

- Next flip is Tails (**50% or $1 - p$**): The node's tower stops at **Level k** .

Why **$1 - p$** ?

Due to calculation, when the probability that it stops at level k :

$$\text{Pr}(\text{Stops at Level } k \mid \text{Already on Level } k) = \frac{p^k(1 - p)}{p^k} = 1 - p = 0.5$$

Where , **pr** represents **probability** $\left(\text{i. e. } \frac{\text{Probability when stops at level } k}{\text{Probability when we are Already standing at level } k} \right)$

Why 50% or $1 - p$ on failing to reach Level $k + 1$ and why 50% or p on successful to reach Level $k + 1$ generates both 0.5 ?

Both generate 0.5 because standard Skip Lists are designed around a fair 50/50 coin toss, where p is chosen to be 0.5.

Here is the exact math and code reason:

1. The Rule of Total Probability

For any trial, there are only two possible outcomes that must add up to 1 (100%):

- *Success (Promoted to Level $k + 1$): Probability = p*
- *Failure (Stops at Level k): Probability = $1 - p$*

$$\text{Success} + \text{Failure} = p + (1 - p) = 1$$

2. Why Both Equal 0.5

Because we explicitly chose $p = 0.5$ (50% chance):

- *Success:*

$$p = 0.5 \quad (50\%)$$

- *Failure:*

$$1 - p = 1 - 0.5 = 0.5 \quad (50\%)$$

0.5 is the only number in mathematics where the probability of success equals the probability of failure ($p = 1 - p$).

3. In our Code (`rand() % 2`)

In the line:

```
while ((rand() % 2) == 0 && lvl < MAX_LEVEL)
```

- *Even numbers (0): `rand()` produces an even number half the time $\Rightarrow p = 0.5$ (Success / Move UP).*
- *Odd numbers (1): `rand()` produces an odd number half the time $\Rightarrow 1 - p = 0.5$ (Failure / Move LEFT).*

4. What If a Different Probability Was Used?

If we designed the Skip List with a 1 – in – 4 chance ($p = 0.25$):

- *Success (p): 0.25 (25%) $\rightarrow$ Move UP*
- *Failure ($1 - p$): $1 - 0.25 = 0.75$ (75%) $\rightarrow$ Move LEFT*

They are only both 0.5 when a standard fair 50/50 coin toss is used.

When we analyze **BACKWARD (Reverse)**:

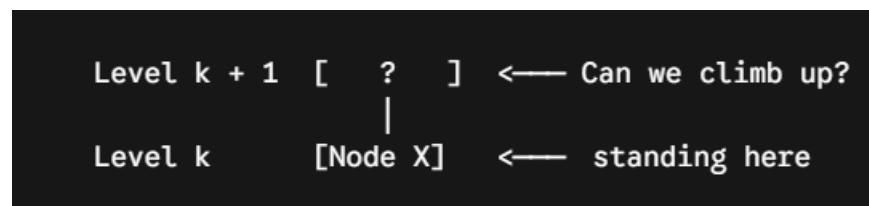
- *Whether a step goes UP or LEFT depends only on the coin flip (`randomLevel()`) of the node you are standing on:*
 - *Promotion succeeded? $\rightarrow$ Move UP (probability p).*
 - *Promotion failed? $\rightarrow$ Move LEFT (probability $1 - p$).*
- *It does not depend on the keys or the target at all.*
- *Every step is an independent coin toss, allowing us to directly use the simple Geometric Distribution formula $\left(\mathbb{E}[X] = \frac{1}{p}\right)$.*

Why Move Left ?

1. What we check while standing at Node X on Level k?

Imagine we are rewinding the search path and you are currently standing at Node X on Level k.

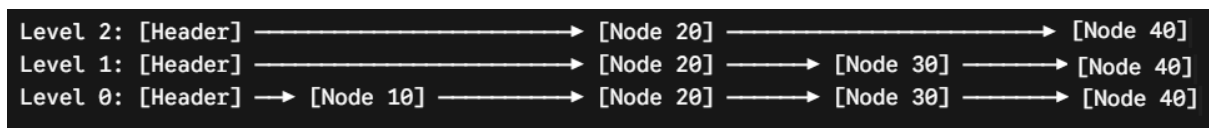
Now Question: "Is there a floor above me on Node X (Level k + 1)?"



2. If Promotion FAILED ($1 - p$):

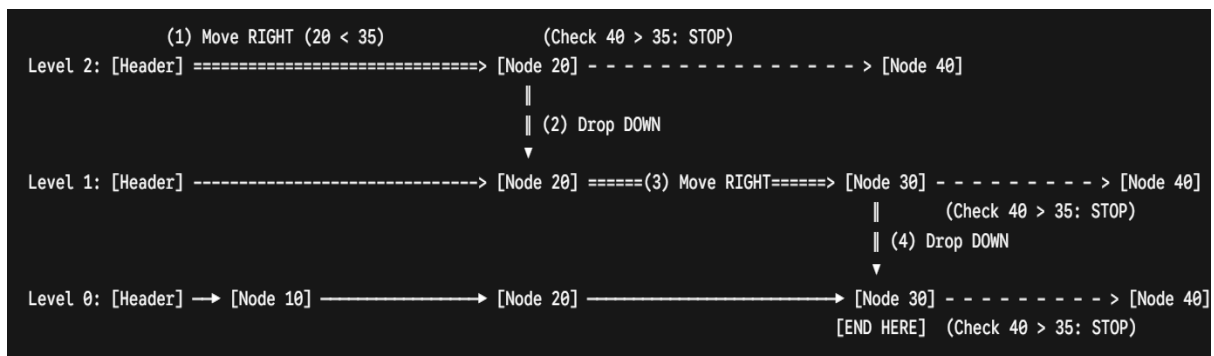
- *The fact: Node X was not promoted to Level k + 1.*
There is no floor above on Node X.
- *The deduction: Since there is no floor above on Node X, the forward search could not have dropped down from above into Node X.*
- *The only possibility: The forward search had to arrive by stepping RIGHT from its left neighbor (Node W) on Level k.*
- *The reverse action: To rewind that step, we must move LEFT to Node W.*

Lets take example of:



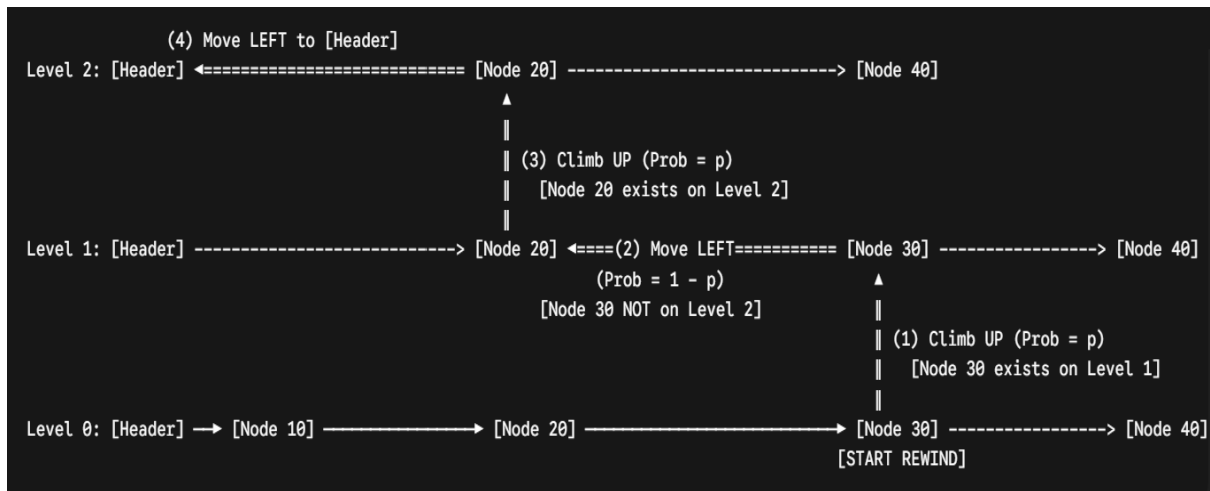
Part 1: The Forward Search (Start → End)

1. ***At Level 2:*** Start at [Header].
 - Move **RIGHT** to [Node 20] ($20 < 35$).
 - Look ahead to [Node 40]: $40 > 35$ (too big).
 - **Drop DOWN** at Node 20 to **Level 1**.
2. ***At Level 1:*** You are at [Node 20].
 - Move **RIGHT** to [Node 30] ($30 < 35$).
 - Look ahead to [Node 40]: $40 > 35$ (too big).
 - **Drop DOWN** at Node 30 to **Level 0**.
3. ***At Level 0:*** You are at [Node 30].
 - Look ahead to [Node 40]: $40 > 35$. Cannot move right.
 - **Search ends at (Node 30, Level 0).**



Part 2: The Backward Walk (End → Start)

Now we retrace this exact path in reverse, starting from (Node 30, Level 0):



Step 1: Standing at (Node 30, Level 0) ($k = 0$)

- **Ask:** Does Node 30 exist on Level 1 ($k + 1$)?
- **Look at the diagram:** YES, [Node 30] has a box on Level 1 (its coin flip succeeded with probability p).
- **Decision:** Climb UP $\Rightarrow$ Now at (Node 30, Level 1).

Step 2: Standing at (Node 30, Level 1) ($k = 1$)

- **Ask:** Does Node 30 exist on Level 2 ($k + 1$)?
- **Look at the diagram:** NO, look above Node 30 on Level 2 – there is no box there! (Its coin flip to Level 2 failed with probability $1 - p$).
- **Decision:** Since there is no floor above to climb, you must move **LEFT** on Level 1 $\Rightarrow$ Now at (Node 20, Level 1).

Step 3: Standing at (Node 20, Level 1) ($k = 1$)

- **Ask:** Does Node 20 exist on Level 2 ($k + 1$)?
- **Look at the diagram:** YES, [Node 20] has a box on Level 2 (its coin flip succeeded with probability p).
- **Decision:** Climb UP $\Rightarrow$ Now at (Node 20, Level 2).

Step 4: Standing at (Node 20, Level 2) ($k = 2$)

- Move **LEFT** to [Header] on Level 2.
- **Rewind complete!** We reached the top – left starting position.

Things we must remember of this backward walk:

*A **purely theoretical mathematical proof technique** on paper used to analyze and calculate expected search complexity .It is never an actual function or algorithm executed by the computer.*

RandomLevel() viewed in program code execution vs Average case math.

1. In the Program (Once per Node)

- When a node is created, `randomLevel()` is called **only once**.
- That single call flips the coins and sets the permanent height (`rlevel`) for that node.
- Once inserted, the node's height never changes.

2. In the Average – Case Analysis (The Math)

Because the program uses a randomized algorithm:

- Every node's height was determined by its own independent call to `randomLevel()`.
- When we calculate the **average (expected) search time** across all nodes in the list:
 - At any arbitrary node on Level k , the probability that its past `randomLevel()` call reached Level $k + 1$ is $p = 0.5$ (Move **UP**).
 - The probability that its `randomLevel()` stopped at Level k is $1 - p = 0.5$ (Move **LEFT**).

<i>Perspective</i>	<i>How randomLevel() is Viewed</i>
<i>Code Execution</i>	<i>Called once per node during <code>insertElement()</code> to allocate its pointers.</i>
<i>Average – Case Math</i>	<i>Because every node had independent coin flips, each backward step has a 50% chance to go UP and a 50% chance to go LEFT, yielding the Geometric Distribution ($\mathbb{E}[\text{steps}] = 1/p = 2$).</i>

When Node is Inserted



randomLevel() flips coins



Decides the Node's Height (How many boxes it gets)



During Backward Analysis:

- Does it have a box above? (Coin flip was Heads) → Go UP
- No box above? (Coin flip was Tails) → Go LEFT

*We want to find the expected number of leftward steps we must take on any given level before we are finally allowed to move UP. This is a classic **Geometric Distribution**(flipping a biased coin until we get our first `success`).*

*The expected number of trials **$E[X]$** to get the first success(moving UP) in a Geometric Distribution is:*

$$E[X] = \frac{1}{p}$$

How?

First way $\rightarrow$ Through Infinite $\left[\begin{array}{c} \text{Arithmetic Geometric} \\ \text{Progression,} \\ \text{[An Infinite Geometric Series]} \end{array} \right]$

Let X be the number of steps until the first success(moving UP with probability p).

Let $q = 1 - p$ be the probability of a failure (moving Left)

Earlier we have seen During Node Creation and Add Levels :

$\rightarrow$ When returned level 1 , 1st flipped passed generated ` $p$ ` but 2nd flipped failed generates ` $1 - p$ ` $\Rightarrow p(1 - p)$

$\rightarrow$ When returned level 2 , 2 times flipped passed generated ` $p$ ` but 3rd flipped failed generates ` $1 - p$ ` $\Rightarrow p^2(1 - p)$

.

.

$\rightarrow$ When returned level k , k times flipped passed generated ` $p$ ` but $k + 1$ flipped failed generates ` $1 - p$ ` $\Rightarrow p^k(1 - p)$

But during our Backward Search :

Finish in 1 step:

1^{st} node can go UP $\Rightarrow P$

Finish in 2 step:

1^{st} node went LEFT $(1 - p) \times 2^{nd}$ node goes UP $[p] \Rightarrow (1 - p)p$ or qp

Finish in 3 step:

1^{st} LEFT $(1 - p) \times 2^{nd}$ LEFT $(1 - p) \times 3^{rd}$ UP $[p] \Rightarrow (1 - p)^2 p$ or $q^2 p$

Finish in k step:

$(k - 1)$ LEFT moves by 1 UP move $\Rightarrow (1 - p)^{k-1} p$ or $q^{k-1} p$

<i>Process</i>	<i>What keeps it going?</i>	<i>What causes it to STOP?</i>	<i>Formula for step /level k</i>
1. randomLevel() (Node Creation)	Heads $(p) \rightarrow$ keep adding levels	Tails $(1 - p)$ $\rightarrow$ stop adding levels	$p^k(1 - p)$
2. Backward Search (Level Traversal)	Tails $(1 - p)$ $\rightarrow$ keep moving LEFT	Heads (p) $\rightarrow$ STOP and climb UP	$(1 - p)^{k-1} p$

What Happens When $p = 0.5$?

When $p = 0.5$, then $(1 - p) = 0.5$ as well:

- **For randomLevel() at lvl = 1:**

$$p(1 - p) = 0.5 \times 0.5 = 0.25 \quad (25\%)$$

- *For Backward Search at step 2:*

$$(1 - p)p = 0.5 \times 0.5 = 0.25 \quad (25\%)$$

Because $p = 1 - p = 0.5$, the numerical values match, but the roles of Heads and Tails are flipped between the two processes.

→ *Success on 1st step: Probability = p (Value = 1)*

→ *Success on 2nd step: Probability = $q \cdot p$ (Value = 2)*

→ *Success on 3rd step: Probability = $q^2 \cdot p$ (Value = 3)*

→ *Success on $k - th$ step: Probability = $q^{k-1} \cdot p$ (Value = k)*

By the definition of Expected Value $\mathbb{E}[X]$:

$$\mathbb{E}[X] = \sum_{k=1}^{\infty} k \times \Pr(X = k) = p(1 + 2q + 3q^2 + 4q^3 + \dots)$$

Solving the infinite Sum S :

$$\text{Let } S = 1 + 2q + 3q^2 + 4q^3 + \dots$$

Multiply both sides by q :

$$qS = q + 2q^2 + 3q^3 + \dots$$

Subtract qS from S :

$$\begin{aligned} S - qS &= (1 - 0) + (2q - q) + (3q^2 - 2q^2) + (4q^3 - 3q^3) + \dots \\ &= 1 + q + q^2 + q^3 + \dots \end{aligned}$$

Now, general infinite geometric series is:

$$a_1 + a_1r + a_1r^2 + a_1r^3 + \dots$$

$$S_{\infty} = \frac{a_1}{1-r} ; |r| < 1$$

Now , we have :

$$S - qS = 1 + q + q^2 + q^3 + \dots$$

$$S - qS = 1 + 1 \times q + 1 \times q^2 + 1 \times q^3 + \dots$$

$$\text{hence , } a_1 = 1$$

$$\text{common ratio} = r = \frac{\text{any term}}{\text{previous term}}$$

$$r = \frac{q}{1} = q; r = \frac{q^2}{q} = q; r = \frac{q^3}{q^2} = q \therefore r = q.$$

$$S_{\infty} = \frac{1}{1-q}$$

$$\text{or, } S - qS = S_{\infty} = \frac{1}{1-q}$$

$$S(1-q) = \frac{1}{1-q}$$

We, know , $q = 1 - p$, then : $1 - (1 - p) = 1 - 1 + p = p$.

$$S \times p = \frac{1}{p}$$

$$\Rightarrow S = \frac{1}{p^2}$$

Now we know:

$$\mathbb{E}[X] = \sum_{k=1}^{\infty} k \times \Pr(X = k) = p(1 + 2q + 3q^2 + 4q^3 + \dots)$$

$$\text{or, } \mathbb{E}[X] = \sum_{k=1}^{\infty} k \times \Pr(X = k) = p \times S$$

Now, substitute S back into $\mathbb{E}[X]$:

$$\mathbb{E}[X] = \sum_{k=1}^{\infty} k \times \Pr(X = k) = p \times \frac{1}{p^2} = \frac{1}{p}.$$

$$\text{Hence, } \mathbb{E}[X] = \frac{1}{p}.$$

2nd Way → First Step Analysis(Intuitive)

We can also solve it by looking at just the very first step:

- 1. With probability p , the first step succeeds (we move UP), We took 1 step.**
- 2. With probability $1 - p$, the first step fails (we move LEFT), We took 1 step, and from this new node, we still need an Expected $\mathbb{E}[X]$ steps.**

$$\mathbb{E}[X] = \text{Pr}(\text{Success}) \times (\text{Steps if Success}) + \text{Pr}(\text{Failure}) \times (\text{Steps if failure})$$

Let $\mathbb{E}[X]$ be the average total steps needed to get our first success (moving UP).

When we take the 1st step, only two mutually exclusive outcomes can happen:

1. Scenario A: The 1st Step is a Success (Probability = p)

- *We flipped Heads and moved UP on our very first try.*
- *The search on this level is finished !*
- *Total steps taken: Exactly 1 step.*
- *Contribution to expectation:*

$$p \times (1)$$

2. Scenario B: The 1st Step is a Failure (Probability = $1 - p$)

- *We flipped Tails and moved Left to the neighbor node.*
- *We have already used up 1 step.*
- *From this new neighbor node, because the coin flips have no memory (memoryless property), our expected number of remaining steps to reach the top is still $\mathbb{E}[X]$.*
- *Total Steps taken in this scenario:*

$$\underbrace{1}_{\substack{\text{1st Step} \\ \text{already} \\ \text{taken}}} + \underbrace{\mathbb{E}[X]}_{\substack{\text{Expected Steps} \\ \text{from the} \\ \text{new node}}} = 1 + \mathbb{E}[X]$$

- *Contribution to expectation:*

$$(\mathbf{1} - \boldsymbol{p}) \times (\mathbf{1} + \mathbb{E}[X])$$

Set up the equation:

$$\mathbb{E}[X] = \boldsymbol{p}(\mathbf{1}) + (\mathbf{1} - \boldsymbol{p}) \times (\mathbf{1} + \mathbb{E}[X])$$

Expand:

$$\mathbb{E}[X] = \boldsymbol{p} + (\mathbf{1} - \boldsymbol{p}) + (\mathbf{1} - \boldsymbol{p}) \times \mathbb{E}[X]$$

$$\mathbb{E}[X] = \mathbf{1} + (\mathbf{1} - \boldsymbol{p}) \times \mathbb{E}[X]$$

Subtract $(\mathbf{1} - \boldsymbol{p}) \mathbb{E}[X]$ from both sides:

$$\mathbb{E}[X] - \{(\mathbf{1} - \boldsymbol{p}) \mathbb{E}[X]\} = \mathbf{1} + \{(\mathbf{1} - \boldsymbol{p}) \mathbb{E}[X]\} - \{(\mathbf{1} - \boldsymbol{p}) \mathbb{E}[X]\}$$

$$\mathbb{E}[X] - \{(\mathbf{1} - \boldsymbol{p}) \mathbb{E}[X]\} = \mathbf{1}$$

$$\mathbb{E}[X] - \{\mathbb{E}[X] - \boldsymbol{p} \times \mathbb{E}[X]\} = \mathbf{1}$$

$$\mathbb{E}[X] - \mathbb{E}[X] + \boldsymbol{p} \times \mathbb{E}[X] = \mathbf{1}$$

$$\boldsymbol{p} \times \mathbb{E}[X] = \mathbf{1}$$

$$\mathbb{E}[X] = \frac{\mathbf{1}}{\boldsymbol{p}}$$

If $p = 0.5$, we expect to take exactly 2 steps (comparisons) per level before moving up to the next one.

When $p = 0.5$:

$$\mathbb{E}[X] = \frac{1}{0.5} = \frac{1}{\frac{1}{2}} = 2 \text{ steps.}$$

Extra Calculation:

1. Expected Pointers per Node (Memory Complexity)

When a node is created, how many pointers (forward[]) does it have on average?

- *Every node has Level 0: 1 pointer (always, probability = 1).*
- *It also gets Level 1 with probability: p .*
- *It also gets Level 2 with probability: p^2 .*
- *It also gets Level 3 with probability: p^3 , and so on.*

The expected total number of pointers in a node is the sum of this infinite geometric series:

$$\mathbb{E}[\text{Pointers per Node}] = 1 + p + p^2 + p^3 + \dots = \sum_{i=0}^{\infty} p^i = \frac{1}{1-p}$$

$$\text{if } p = 0.5 (50\%) = \frac{1}{1-0.5} = \frac{1}{1-\frac{1}{2}} = \frac{1}{\frac{1}{2}} = 2 \text{ pointers per node on average.}$$

$$\text{if } p = 0.25(25\%) = \frac{1}{1 - 0.25} = \frac{1}{1 - \frac{1}{4}} = \frac{1}{\frac{3}{4}} = \frac{4}{3} \\ \approx 1.33 \text{ pointers per node on average.}$$

2. Calculation of Expected strictly horizontal (left) steps per level:

1. Problem Setup and Random Variable Definition

Let H be the random variable for the number of strictly horizontal(leftward) steps taken on a level before moving UP.

- Probability of moving UP(Success): P
- Probability of moving Left (Failure and Step Left): $q = 1 - p$

The possible values for H are 0, 1, 2, 3,:

- $H = 0$ (0 left steps): The current node goes UP immediately $\Rightarrow Pr(H = 0) = p$
- $H = 1$ (1 left step): 1 left move, then 1 UP move $\Rightarrow Pr(H = 1) = (1 - p)p$ or qp
- $H = 2$ (2 left steps): 2 left moves, then 1 UP move $\Rightarrow Pr(H = 2) = (1 - p)^2p$ or q^2p .
- .
- .
- .
- $H = (k \text{ left steps})$: k left moves, then 1 UP move $\Rightarrow Pr(H = k) = (1 - p)^k p$ or $q^k p$

Method 1: Direct Infinite Series (AGP) Derivation

By the definition of mathematical expectation:

$$\mathbb{E}[H] = \sum_{k=0}^{\infty} k \cdot Pr(H = k)$$

Since the term for $k = 0$ is $0 \cdot Pr(H = k) = 0$, we can start sum at $k = 1$.

$$\mathbb{E}[H] = \sum_{k=1}^{\infty} k \cdot \text{Pr}(H = k)$$

$$\Rightarrow \mathbb{E}[H] = \sum_{k=1}^{\infty} k \cdot (q^k p)$$

$$\Rightarrow \mathbb{E}[H] = p \sum_{k=1}^{\infty} k q^k$$

Factor out one q :

$$\Rightarrow \mathbb{E}[H] = p \cdot q \sum_{k=1}^{\infty} k q^{k-1}$$

$$\text{Let } S = \sum_{k=1}^{\infty} k q^{k-1} = 1 + 2q + 3q^2 + 4q^3 + \dots$$

Multiply both sides by q :

$$qS = q + 2q^2 + 3q^3 + \dots$$

Subtract qS from S :

$$\begin{aligned} S - qS &= (1 - 0) + (2q - q) + (3q^2 - 2q^2) + (4q^3 - 3q^3) + \dots \\ &= 1 + q + q^2 + q^3 + \dots \end{aligned}$$

Now, general infinite geometric series is:

$$a_1 + a_1 r + a_1 r^2 + a_1 r^3 + \dots$$

$$S_{\infty} = \frac{a_1}{1 - r} ; |r| < 1$$

Now, we have :

$$S - qS = 1 + q + q^2 + q^3 + \dots$$

$$S - qS = 1 + 1 \times q + 1 \times q^2 + 1 \times q^3 + \dots$$

$$\text{hence, } a_1 = 1$$

$$\text{common ratio} = r = \frac{\text{any term}}{\text{previous term}}$$

$$r = \frac{q}{1} = q; r = \frac{q^2}{q} = q; r = \frac{q^3}{q^2} = q \therefore r = q.$$

$$S_{\infty} = \frac{1}{1 - q}$$

$$\text{or, } S - qS = S_{\infty} = \frac{1}{1 - q}$$

$$S(1 - q) = \frac{1}{1 - q}$$

$$\text{We, know, } q = 1 - p, \text{ then : } 1 - (1 - p) = 1 - 1 + p = p.$$

$$S \times p = \frac{1}{p}$$

$$\Rightarrow S = \frac{1}{p^2}$$

$$\text{Substitute } S \text{ back into } \mathbb{E}[H] :$$

$$\mathbb{E}[H] = p \cdot q \sum_{k=1}^{\infty} kq^{k-1}$$

$$\Rightarrow \mathbb{E}[H] = p \cdot q \times S$$

$$\Rightarrow \mathbb{E}[H] = p \cdot q \times \frac{1}{p^2}$$

$$\Rightarrow \mathbb{E}[H] = \frac{q}{p}$$

$$\Rightarrow \mathbb{E}[H] = \frac{1-p}{p}$$

Method 2: Relationship to Total Steps($X = H + 1$)

On any given floor during the backward walk:

- *We take some number of LEFT steps (H).*
- *We finish the floor by taking exactly 1 UP step.*

Therefore, the Total Steps(X) we took on that floor is always:

Total Steps(X) = (Horizontal Steps H) + (1 Vertical Step UP)

$$**X = H + 1**$$

Rearranging the equation to find H :

$$**H = X - 1**$$

<i>What happens on the floor</i>	<i>Sequence of Moves</i>	<i>Total Steps (X)</i>	<i>Horizontal Steps (H)</i>	<i>Check (H = X - 1)</i>
0 Left moves, then 1 UP	[UP]	$X = 1$	$H = 0$	$0 = 1 - 1$
1 Left move, then 1 UP	[LEFT, UP]	$X = 2$	$H = 1$	$1 = 2 - 1$
2 Left moves, then 1 UP	[LEFT, LEFT, UP]	$X = 3$	$H = 2$	$2 = 3 - 1$
3 Left moves, then 1 UP	[LEFT, LEFT, LEFT, UP]	$X = 4$	$H = 3$	$3 = 4 - 1$

In every single case, the number of LEFT steps is simply the total steps minus 1.

$\mathbb{E}$ simply means ``the average value``.

if a quantity H is always $X - 1$, then its average $\mathbb{E}[H]$ must also equal the average of $X - 1$:

$$\mathbb{E}[H] = \mathbb{E}[X] - 1$$

*Now we know from **Backward Walk**:*

$$\mathbb{E}[X] = \sum_{k=1}^{\infty} k \times \text{Pr}(X = k) = p(1 + 2q + 3q^2 + 4q^3 + \dots) = \frac{1}{p}$$

Now substituting this into our formula, we get:

$$\mathbb{E}[H] = \frac{1}{p} - 1$$

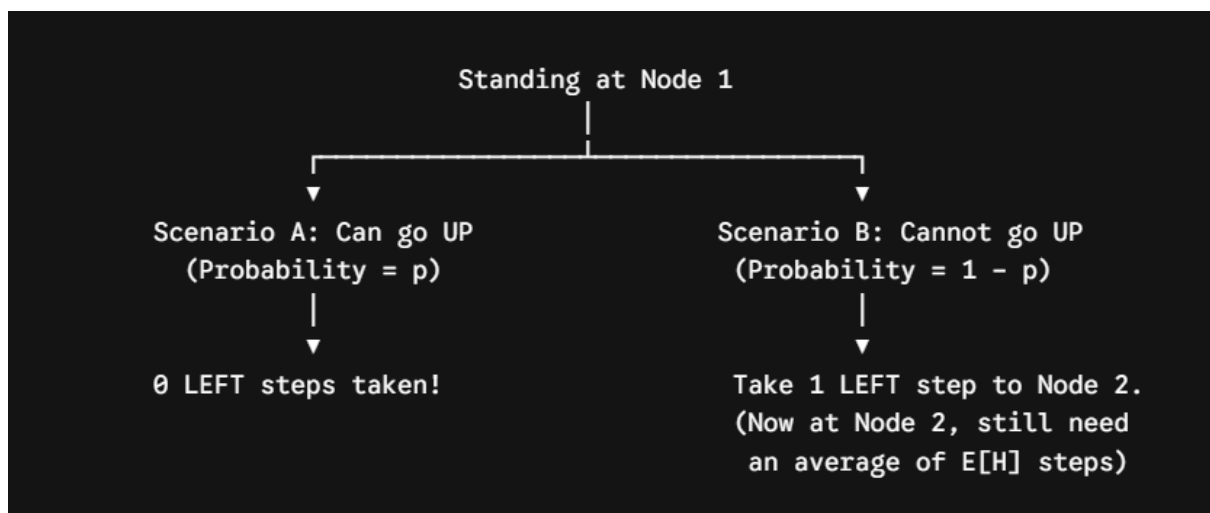
$$\mathbb{E}[H] = \frac{1}{p} - 1$$

$$\Rightarrow \mathbb{E}[H] = \frac{1-p}{p}$$

Method 3: First Step Analysis

Imagine → standing at the starting node on Level k.

if we look at this very first node, and only two mutually exclusive scenarios can happen:



Scenario A: The 1st node can go UP (Probability = p)

- *The node exists on the floor above .*
- *We climb UP immediately.*
- *How many LEFT steps did we take? Exactly 0 horizontal steps.*
- *Contribution to average:*

$$p \times (0)$$

Scenario B: The 1st node can go UP (Probability = $1 - p$)

- *The node does not exist on the floor above.*
- *We are forced to take 1 step LEFT to the neighbor node.*
- *Now we are standing at this new neighbor node. Because coin flips have no memory (memory less property), the expected number of remaining LEFT steps from this new node is still $\mathbb{E}(H)$.*
- *Total LEFT steps taken in this scenario:*

$$\begin{array}{ccc}
 1 & + & \mathbb{E}[H] \\
 \downarrow & & \downarrow \\
 \boxed{\begin{array}{c} \text{the 1 LEFT} \\ \text{step we just} \\ \text{took} \end{array}} & & \boxed{\begin{array}{c} \text{average LEFT} \\ \text{steps remaining} \\ \text{from new node.} \end{array}}
 \end{array}
 = 1 + \mathbb{E}[H]$$

- *Contribution to average:*

$$(1 - p) \times (1 + \mathbb{E}[H])$$

By the Law of Total Expectation, the overall average $\mathbb{E}[H]$ is the weighted sum of both scenarios:

$$\mathbb{E}[H] = \underbrace{p \cdot (0)}_{\text{Scenario A (Climb UP)}} + \underbrace{(1-p) \cdot (1 + \mathbb{E}[H])}_{\text{Scenario B (Step left)}}$$

$$\begin{aligned}\mathbb{E}[H] &= p \cdot (0) + (1-p) \cdot (1 + \mathbb{E}[H]) \\ \Rightarrow \mathbb{E}[H] &= 0 + (1-p) \cdot (1 + \mathbb{E}[H]) \\ \Rightarrow \mathbb{E}[H] &= (1-p) + \mathbb{E}[H](1-p) \\ \Rightarrow \mathbb{E}[H] &= (1-p) + \mathbb{E}[H] - p \cdot \mathbb{E}[H]\end{aligned}$$

Adding $p \cdot \mathbb{E}[H]$ on both the sides we get:

$$\begin{aligned}\Rightarrow \mathbb{E}[H] + p \cdot \mathbb{E}[H] &= (1-p) + \mathbb{E}[H] - p \cdot \mathbb{E}[H] + p \cdot \mathbb{E}[H] \\ \Rightarrow \mathbb{E}[H] + p \cdot \mathbb{E}[H] &= (1-p) + \mathbb{E}[H]\end{aligned}$$

Subtracting $\mathbb{E}[H]$ on both the sides we get:

$$\begin{aligned}\Rightarrow \mathbb{E}[H] + p \cdot \mathbb{E}[H] - \mathbb{E}[H] &= (1-p) + \mathbb{E}[H] - \mathbb{E}[H] \\ \Rightarrow p \cdot \mathbb{E}[H] &= (1-p) \\ \Rightarrow \mathbb{E}[H] &= \frac{1-p}{p}\end{aligned}$$

- *Total steps on average:* $\mathbb{E}[X] = \frac{1}{0.5} = \frac{1}{\frac{1}{2}} = 2$ *total steps* ([1 LEFT + 1 UP])
- *Horizontal Only Steps :* $\mathbb{E}[H] = \frac{1-0.5}{0.5} = \frac{1-\frac{1}{2}}{\frac{1}{2}} = \frac{\frac{1}{2}}{\frac{1}{2}}$
 $= 1$ *horizontal step on average per level* [1 LEFT]

Now coming to back to the calculation of Average Case Complexity:

Step 1C: Combine for Total Search Cost

The total expected number of steps in the search path is the number of levels multiplied by the expected steps taken on each level:

$$\text{Expected Search Cost} = H \cdot \mathbb{E}[X]$$

$$\text{Expected Search Cost} = \left(\log_{\frac{1}{p}} n \right) \cdot \left(\frac{1}{p} \right)$$

Evaluating for $p = 0.5$:

$$\begin{aligned} &= \left(\log_{\frac{1}{0.5}} n \right) \cdot \left(\frac{1}{0.5} \right) \\ &= \left(\log_{\frac{1}{\frac{1}{2}}} n \right) \cdot \left(\frac{1}{\frac{1}{2}} \right) \\ &= (\log_2 n) \cdot (2) \end{aligned}$$

Because $\frac{1}{p}$ is a constant factor, we drop it for asymptotic notation, leaving us with a search time complexity of $O(\log_2 n)$.

Therefore, for Phase 1 : Time Complexity is $O(\log_2 n)$.

Phase 2: The Coin Flip Phase (Level Generation)

Once the search is complete, the algorithm generates a random level for the newly inserted node using a while loop that flips a coin.

- The probability of stopping the loop (getting a "tails") is $1 - p$.
- This is another Geometric Distribution.

Recall:

```
int randomLevel() {  
    int lvl = 0;  
    // 50% probability to increase the level  
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {  
        lvl++;  
    }  
    return lvl;  
}
```

Returned lvl	Formula	Reason	Probability
lvl = 0	$1 - p$	rand()%2 = 1, very first coin flipped failed.	50% = 0.5
lvl = 1	$p(1 - p)$	rand()%2 = 1, at 2nd time coin flipped failed.	50% × 50% = 0.5² = 0.25 = 0.25 × 100 = 25%
lvl = 2	$p^2(1 - p)$	rand()%2 = 1, at 3rd time coin flipped failed.	50% × 50% × 50% = 0.5³ = 0.125 = 0.125 × 100 = 12.5%
lvl = 3	$p^3(1 - p)$	rand()%2 = 1, at 4th time coin flipped failed.	50% × 50% × 50% × 50% = 0.5⁴ = 0.0625 = 0.0625 × 100 = 6.25%

- $N = 1$ flip(stops immediately on 1st flip):

$$\Pr(N = 1) = (1 - p)$$

- $N = 2$ flips(1 Heads, then 1 Tails):

$$\Pr(N = 2) = p(1 - p)$$

- $N = 3$ flips(2 Heads, then 1 Tails):

$$\Pr(N = 3) = p^2(1 - p)$$

- $N = k$ flips($k - 1$ Heads, then 1 Tails):

$$\Pr(N = k) = p^{k-1}(1 - p)$$

By the definition of Expected Value $\mathbb{E}[N] = \sum_{k=1}^{\infty} k \cdot \Pr(N = k)$:

$$\Rightarrow \mathbb{E}[N] = 1 \cdot (1 - p) + 2 \cdot p(1 - p) + 3 \cdot p^2(1 - p) + 4 \cdot p^3(1 - p) + \dots$$

$$\Rightarrow \mathbb{E}[N] = (1 - p) \sum_{k=1}^{\infty} k \times p^{k-1}$$

$$\Rightarrow \mathbb{E}[N] = (1 - p)(1 + 2p + 3p^2 + 4p^3 + \dots)$$

$$\text{Let, } S = \sum_{k=1}^{\infty} k \cdot p^{k-1} = 1 + 2p + 3p^2 + 4p^3 + \dots$$

$$\therefore \mathbb{E}[N] = (1 - p) \times S$$

Now ,Evaluating the Infinite Series `S`:

$$\text{Let } S = \sum_{k=1}^{\infty} kp^{k-1} = 1 + 2p + 3p^2 + 4p^3 + \dots$$

Multiply both sides by p:

$$pS = p + 2p^2 + 3p^3 + \dots$$

Subtract pS from S:

$$\begin{aligned} S - pS &= (1 - 0) + (2p - p) + (3p^2 - 2p^2) + (4p^3 - 3p^3) + \dots \\ &= 1 + p + p^2 + p^3 + \dots \end{aligned}$$

Now, general infinite geometric series is:

$$a_1 + a_1r + a_1r^2 + a_1r^3 + \dots$$

$$S_{\infty} = \frac{a_1}{1-r} ; |r| < 1$$

Now ,we have :

$$S - pS = 1 + p + p^2 + p^3 + \dots$$

$$S - pS = 1 + 1 \times p + 1 \times p^2 + 1 \times p^3 + \dots$$

$$\text{hence , } a_1 = 1$$

$$\text{common ratio} = r = \frac{\text{any term}}{\text{previous term}}$$

$$r = \frac{p}{1} = p; r = \frac{p^2}{p} = p; r = \frac{p^3}{p^2} = p \therefore r = p.$$

$$S_{\infty} = \frac{1}{1-p}$$

$$\text{or, } S - pS = S_{\infty} = \frac{1}{1-p}$$

$$S(1-p) = \frac{1}{1-p}$$

$$\Rightarrow S = \frac{1}{(1-p)^2}$$

Final Substitution:

$$\begin{aligned} \mathbb{E}[N] &= (1-p) \times S \\ \Rightarrow \mathbb{E}[N] &= (1-p) \times \frac{1}{(1-p)^2} \end{aligned}$$

Cancel $(1-p)$ from numerator and denominator:

$$\mathbb{E}[N] = \frac{1}{1-p}$$

$$\Rightarrow \mathbb{E}[\text{flips}] = \frac{1}{1-p}$$

Even if we calculate from the Table:

$$\mathbb{E}[\text{Coin Flips}] = \sum (\text{Flips}) \times \sum (\text{Probability from the table})$$

$$\mathbb{E}[\text{Coin Flips}] = \underbrace{1 \cdot (0.5)}_{\text{for lvl} = 0} + \underbrace{2 \cdot (0.25)}_{\text{for lvl} = 1} + \underbrace{3 \cdot (0.125)}_{\text{for lvl} = 2} + \underbrace{4 \cdot (0.0625)}_{\text{for lvl} = 3} + \dots$$

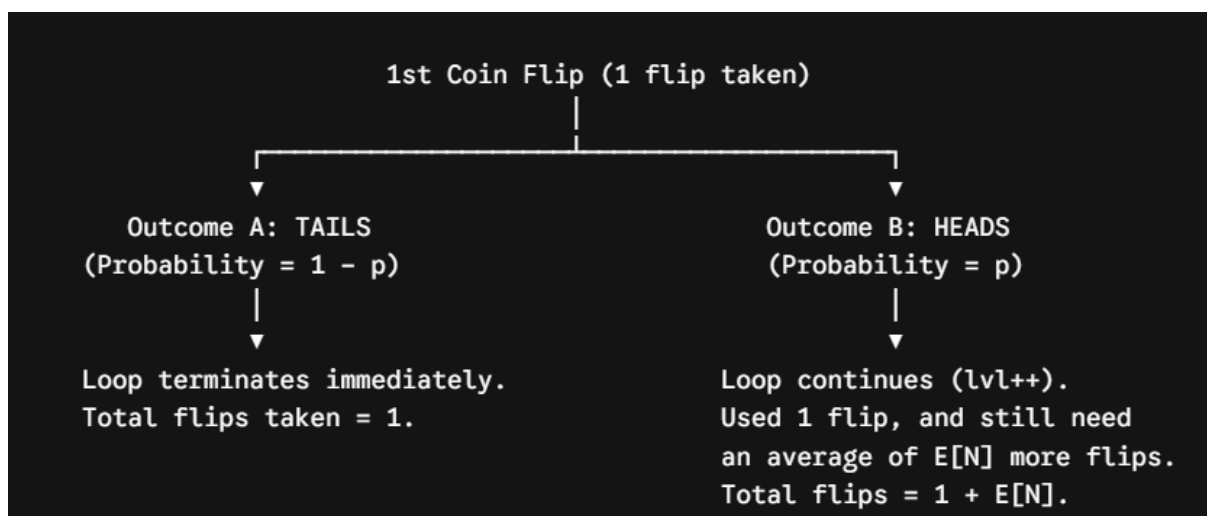
$$\mathbb{E}[\text{Coin Flips}] = 0.5 + 0.5 + 0.375 + 0.25 + \dots = 2 \text{ flips.}$$

This matches the closed – form formula:

$$\Rightarrow \mathbb{E}[\text{flips}] = \frac{1}{1-p} = \frac{1}{1-0.5} = 2 \text{ flips.}$$

Through Method 2: First Step Analysis

Let $\mathbb{E}[N]$ be the expected total number of coin flips inside `randomLevel()` until the loop terminates.



Outcome A: The 1st flip is TAILS (Probability = $1 - p$)

- *The while condition is false. The loop exists immediately.*
- *Total flips taken: Exactly 1 flip.*
- *Contribution to expectation:*

$$(1 - p) \cdot (1)$$

Outcome B: The 1st flip is Heads (Probability = p)

- *The while condition is true, `lvl++` executes, and the loop prepares for the next flip.*
- *We have spent 1 flip.*
- *By the memoryless property of independent coin tosses, the expected number of remaining flips needed from this point onward is still $\mathbb{E}[N]$.*
- *Total flips taken in this scenario:*

$$1 + \mathbb{E}[N]$$

- *Contribution to expectation:*

$$p \cdot (1 + \mathbb{E}[N])$$

By the law of Total Expectation:

$$\mathbb{E}[N] = \underbrace{(1-p) \cdot (1)}_{\text{If 1st flip is Tails}} + \underbrace{p \cdot (1 + \mathbb{E}[N])}_{\text{If 1st flip is Heads}}$$

$$\Rightarrow \mathbb{E}[N] = 1 - p + p + p \cdot \mathbb{E}[N]$$

$$\Rightarrow \mathbb{E}[N] = 1 + p \cdot \mathbb{E}[N]$$

Subtract $p \cdot \mathbb{E}[N]$ from both sides to group $\mathbb{E}[N]$ on the left:

$$\Rightarrow \mathbb{E}[N] - p \cdot \mathbb{E}[N] = 1$$

$$\Rightarrow \mathbb{E}[N](1 - p) = 1 \text{ [Factor out } \mathbb{E}[N] \text{]}$$

Divide both sides by $(1 - p)$:

$$\Rightarrow \mathbb{E}[N] = \frac{1}{1 - p}$$

$$\Rightarrow \mathbb{E}[flips] = \frac{1}{1 - p}$$

Results for $p = 0.5$

$$\Rightarrow \mathbb{E}[flips] = \frac{1}{1 - 0.5}$$

$$\Rightarrow \mathbb{E}[flips] = \frac{1}{1 - \frac{1}{2}}$$

$$\Rightarrow \mathbb{E}[flips] = \frac{1}{\frac{1}{2}}$$

$$\Rightarrow \mathbb{E}[flips] = 2$$

If $p = 0.5$, the algorithm expects to flip the coin exactly 2 times. Because this is a strict mathematical constant regardless of how large n grows, this phase takes $O(1)$ expected time.

Phase 3: The Splicing Phase (Pointer Rewiring)

1. Mechanism: The algorithm loops from level 0 up to $rlevel$, using the saved `update[ ]` array to splice the new node into each linked list:

```
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
```

Average Cost: The number of levels rewired is **$rlevel + 1$** .

As proved in **Phase 2**, the expected number of pointers per node is a constant:

$$\mathbb{E}[Pointers] = \frac{1}{1 - p} = 2 \text{ (for } p = 0.5 \text{)}$$

Thus, pointer rewiring runs in $O(1)$ expected time.

2. Worst – Case Upper Bound $\rightarrow O(\log n)$

1. The Maximum Level Cap:

- If a node gets exceptionally lucky and flips consecutive Heads, $rlevel$ can grow up to the maximum height of the list:

$$MAX_LEVEL = \log_{\frac{1}{p}} n$$

$$p = 0.5, \text{ then } MAX_LEVEL = \log_2 n \quad \left[\begin{array}{l} \text{Already proved} \\ \text{from Phase 1} \end{array} \right]$$

- In that worst – case scenario, the for loop will run $\log_2 n$ times, performing $O(\log_2 n)$ operations.

2. List Level Extension:

- If $rlevel > lst \rightarrow level$, the code runs an extra loop to point new levels to the Header:

```
if (rlevel > lst->level) {  
    for (int i = lst->level + 1; i <= rlevel; i++) {  
        update[i] = lst->header;  
    }  
    lst->level = rlevel;  
}
```

This is also strictly bounded by $MAX_LEVEL = \log_{\frac{1}{p}} n$

$$p = 0.5, \text{ then } MAX_LEVEL = \log_2 n \quad \left[\begin{array}{l} \text{Already proved} \\ \text{from Phase 1} \end{array} \right]$$

- In that worst – case scenario, the for loop will run $\log_2 n$ times, performing $O(\log_2 n)$ operations.

Therefore, the splicing phase requires an average of $O(1)$ operations, strictly bounded by a worst – case of $O(\log_2 n)$ operations.

*Since $O(1) \subseteq O(\log n)$, hence we keep the upper bound $O(\log_2 n)$ for **Phase3 : Splicing Phase(Pointer Rewiring)**.*

Final Conclusion

To find the average case time complexity for the entire insertion module, we sum the mathematical bounds of the three phases:

$$\text{Search: } O(\log_2 n) + \text{Coin Flip: } O(1) + \text{Splicing: } O(\log_2 n) = O(\log_2 n)$$

$$\text{Total Time} = O(\log_2 n)$$

Through geometric probability distributions and expected value derivations, the mathematics prove that the average case time complexity for inserting an element into a skip list is strictly $O(\log_2 n)$.

And we get $O(\log_2 n)$, when $p = 0.5$ falls in $O(\log n)$ class.

And both notations represent the exact same complexity class.

$$O(\log_2 n) \approx O(\log n).$$
